Pasted text.txtDocument

based on the following input, help me to create a working drafter site that will run my game. It MUST meet all of these requirements:

Below are the requirements that your site must fulfill. Plan ahead so that meeting these requirements is easier as you develop your project.

- At least 7 routes.
- At least 5 Pages, or 10 FragmentsLinks to an external site., or a combination thereof.
- The implementation of the *State* dataclass must have at least 4 fields.
    - At least one field must be a list of class instances or a dictionary of class instances.
        - This means that there must be a second dataclass.
    - At least 4 state fields, including the list or dictionary of class instances, must each be meaningfully modified in at least one route; those 4 state fields cannot be constants.
- At least 3 input fields (TextBox, CheckBox, SelectBox, TextArea). They don't have to be the same type.
- At least 3 meaningful if statements.
- At least 1 loop that iterates over an attribute of the list of class instances or dictionary of class instances in a meaningful way.
- The site should have a legitimate purpose or functionality.
- No global variables (are fine).
- You must use the Drafter library.
- Any advanced coding techniques (i.e. that weren't covered in this course) used are thoroughly documented in the code.

More creative and impressive designs are given flexibility on the above requirements - impress us! However, impressive websites tend to do more than the above requirements. Further, if you miss more than 2 requirements, you will get no credit for that submission. For example, missing a second dataclass means you would be missing not only that requirement, but also the list or dictionary of such dataclass instances, and the loop iterating over that list. This would mean that the submission would get a zero.

Code Requirements

Make sure your program and website runs; broken programs do not earn points. This means that if your program contains a syntax error or runs and doesn't deploy a website, you get zero points for this assignment, and are subject to any other penalties listed in .

Once you are finished with your project code, check over the following:

- Chosen good variable names (check last two sections of)
- Unit tested your functions where possible, being : each route must have at least one unit test, if possible (routes that have randomization are difficult to unit test, so they are not required to).
    - If a function can't be unit tested, you must note as much in a docstring or comment.
- Broke up excessively long lines (80 characters is a good rule of thumb, but we'll be strict about not exceeding 120 characters)
- Included types in your function headers
- Included good as the first line of your function's body
- Decomposed complicated functions with helper functions

here is the outline for my code:

…

Your project idea is a timed mental-math game where players choose a difficulty, answer randomly generated math questions, build streaks, earn points, and compare their results on a leaderboard.

I recommend keeping your original seven-page design. It already fits the assignment requirements very well. I'll build on your planning document rather than changing the purpose of your project.

There is one important technical detail: standard Drafter does not automatically provide a live countdown timer. We can still implement your timer setting and a functional "I ran out of time" button, which records the question as incorrect and resets the streak. This avoids adding unsupported JavaScript or complicated techniques.

1. How your finished website will work

MATH RUSH

YOUR GAME'S PAGE STRUCTURE

Home Page

Start Game · Instructions · Leaderboard

Instructions Page

Learn how to play

Settings Page

Choose difficulty, rounds, and timer

Game Page

Answer questions · Earn points · Build streaks

Result Page

See whether your answer was correct

Summary Page

Final score · Accuracy · Highest streak

Leaderboard Page

Best score · Best streak · Previous games

The site will also have additional routes for submitting answers, handling timeouts, and starting another round.

2. How the code will meet your requirements

Assignment requirement	Implementation
At least 7 routes	10 routes
At least 5 Pages	7 main pages
Two dataclasses	`State` and `Question`
At least 4 State attributes	10 attributes
List of dataclass instances	`questions: list[Question]`
At least 3 inputs	Difficulty, rounds, timer, answer
At least 3 if statements	Difficulty, correctness, timeout, game completion
Loop through dataclass list	Accuracy and streak calculations
Legitimate functionality	Playable math game
Unit testing	Bakery `assert_equal` tests

3. Your complete Python program

math_rush.py

Complete Drafter project · 480 lines
Syntax checked

Download your complete Math Rush program

The file includes:

- All 11 routes.
- Three dataclasses: `State`, `Question`, and `GameRecord`.
- Randomly generated math questions.
- Three difficulty levels.
- Score and streak tracking.
- Game summaries and a leaderboard.
- Bakery unit tests.
- Docstrings explaining the functions.

4. How to get your website running

1. Download `math_rush.py`.
2. Open your Drafter programming environment.
3. Copy the contents into your project file (usually `main.py`).
4. Make sure Drafter and Bakery are available in your environment.
5. Run the program.

The final line:

```
start_server(initial_state)
```

starts your website.

I have checked the Python syntax, but you should still run it in your course's Drafter environment to verify the website and its tests.

5. Important assignment checklist

Designed to meet all listed requirements

Requirement	Where to find it
7+ routes	All `@route` functions
5+ pages	Seven main pages
4+ State attributes	`State` dataclass
Second dataclass	`Question`
List of instances	`questions`
Meaningful state changes	`start_game`, `submit_answer`, `timeout`, etc.
3+ inputs	Three `SelectBox` inputs and one `TextBox`
3+ conditionals	Throughout the game logic
Loop through dataclass instances	`calculate_accuracy`, `calculate_highest_streak`
No global variables	Only global constants

Unit testing	`assert_equal` tests

One thing to remember: The leaderboard stores results while the current website session is running. It does not permanently save results after the server restarts.

6. My recommended next experiment

Before submitting, try playing a complete five-question game:

- Choose easy difficulty.
- Answer three questions correctly.
- Answer one incorrectly.
- Use the timeout button on the final question.
- Check whether your score, streak, accuracy, and leaderboard update correctly.

This is particularly important because it tests several of your assignment requirements at once.

You have a solid project foundation here. The most important thing now is to run it, test each route, and make sure you understand how the functions work so you can confidently explain your project to your instructor.

math_rush.pyCode

so a timer is not possible in drafter?

A timer is possible in Drafter, but there are limitations depending on what kind of timer you want.

For your Math Rush game, there are three possibilities:

Timer type	Possible?	How it works
Time limit setting	Yes	Player chooses 10, 15, 30, or 60 seconds.

Automatic countdown	Not natively	Drafter does not provide a built-in live countdown component.
Automatic timeout	Requires extra work	Requires additional timing functionality beyond ordinary Drafter routes.

My recommendation: Keep the timer selection in your project, but make it an actual time allowance rather than claiming it is a live countdown.

For example:

1. Player selects 15 seconds.
2. The game displays "You have 15 seconds to answer."
3. Player submits an answer.
4. If they click "I Ran Out of Time," the question is marked incorrect and their streak resets.

One important distinction: this version does not automatically detect when 15 seconds have passed. The player manually reports the timeout.

If you want a genuinely automatic countdown, we would need additional browser-side JavaScript or another timing solution beyond standard Drafter components.

I would recommend the simpler version for your CS1 final project. It keeps your code understandable and avoids introducing advanced techniques that you may not have covered in class.

1. Player selects 15 seconds.
2. The game displays "You have 15 seconds to answer."
3. Player submits an answer.

4. If they click "I Ran Out of Time," the question is marked incorrect and their streak resets.

implement this into the code and update

I can make this update, but data analysis (Python) is temporarily unavailable. It may become available again in about 24 hours, though that timing isn't guaranteed. I wasn't able to save the revised file, so the existing `math_rush.py` remains unchanged.

The good news is that your existing code already has the “I Ran Out of Time” button and the timeout behavior that marks the question incorrect and resets the streak. The remaining changes are:

1. Change the time message in `game_page` to:

```
"You have " + str(state.timer_limit) + " seconds to answer.",
```

2. Keep the existing timeout button:

```
Button("I Ran Out of Time", "timeout"),
```

3. Your existing `timeout` route already handles the timeout:

```
@route
def timeout(state: State) -> Page:
    """Record that the player ran out of time on the current question."""
    if len(state.questions) == 0:
        return settings_page(state)

    current_question = state.questions[-1]
    current_question.user_answer = "Timed out"
```

```
    current_question.correct = False
    current_question.time_used = state.timer_limit
    state.streak = 0


    return result_page(state)
```

Important: This is a player-reported timeout, not an automatic countdown. The game displays the selected time allowance, and the player clicks the timeout button when time runs out.

this code only displays hello world when run in thonny

That sounds like the wrong file is being run, or the Drafter program isn't being started correctly. Let's figure out what's happening.

The Math Rush program I created is a Drafter web application, not a regular program that prints its pages in Thonny's Shell.

Step 1: Check which file you're running

In Thonny:

1. Open `math_rush.py` (or the updated file, if you have one).
2. Look at the first few lines. You should see:

```
from dataclasses import dataclass
from random import randint
from bakery import assert_equal
from drafter import *
```

3. Scroll to the very bottom. You should see something similar to:

```
start_server(initial_state)
```

Step 2: Run the web application

1. Make sure Drafter is installed in the Python environment you're using.
2. Click Run in Thonny.
3. Look in the Shell for the server address.
4. Open that address in your web browser.

You should see the Math Rush home page in the browser, not in the Thonny Shell.

Step 3: Let's check your actual file

Can you upload the exact `.py` file you're running in Thonny?

I can inspect it and identify why you're seeing only "Hello World." I don't want to give you another version that has the same problem.